

ASI.BASECODE Reviewer Guide

This document defines the required architecture and coding rules for ``wigglebop25/AttendTa``. Reviewers should use this as the baseline when checking new code.

1) ASI.BASECODE.DATA

Purpose

``ASI.BASECODE.DATA`` is for database connectivity and persistence operations.

Required rules

- Keep database connection handling and SQL/ORM queries in this layer.
- Do **not** place business logic here (no decision-heavy ``if`/`else`` application rules).
- Repository implementations must focus on data access, not service/application behavior.
- Interfaces define repository contracts/patterns and are used by higher layers (for example, user repository implementations).
- Repository files should stay minimal and straightforward: retrieve, save/add, and update.
- When adding a new domain repository, create a dedicated table for that domain. Do not overload the ``User`` table with unrelated data.
- Models represent database tables.
- Follow **database-first** workflow:
 1. Create the table manually in SQL Server.
 2. Scaffold the table into code.
- SQL Server is required for this table-first process.
- Path management and Unit of Work are out of scope for this reviewer guide and do not need to be studied here.
- Use constants/enums for resources and avoid magic strings. Define values explicitly.

2) ASI.BASECODE.RESOURCE

Purpose

``ASI.BASECODE.RESOURCE`` is the location for shared constants and enums.

Required rules

- Put constants and enums in this area.
- Additional resource files may be allocated for localization (for example Japanese and Korean).

3) ASI.BASECODE.SERVICE

Purpose

``ASI.BASECODE.SERVICE`` contains application/service logic.

Required rules

- Place business and service logic in this layer.
- Preserve the encryption behavior in ``PasswordManager.cs``; encryption is critical to this project. Any change must be necessary, deliberate, and reviewed carefully.
- ``ServiceModels`` should hold service-level models and DTOs.

4) ASI.BASECODE.WEBAPP

Purpose

`ASI.BASECODE.WEBAPP` connects backend logic to the frontend through MVC.

Required rules

- Controllers should stay clean and simple.
- Controllers handle endpoint entry points (GET, POST, etc.) and delegate complex processing to service/job files.
- View models (for example `LoginViewModel.cs`) should stay flexible for frontend requirements.
- Organize Views by screen, with one folder per screen.

5) General project rule

- Do **not** introduce another framework. The project approach and teaching scope rely on the current stack, and adding a new framework adds unnecessary complexity and time.

Recommended request/data flow (short reference)

1. **WebApp Controller** receives request (GET/POST).
2. Controller delegates behavior to a **Service**.
3. **Service** applies business logic and calls **Data repositories** through interfaces.
4. **Data layer** executes persistence operations against SQL Server tables.
5. Response DTO/view model is returned from Service to Controller, then to View/UI.

Reviewer checklist

- [] Data-layer files contain only persistence-focused operations.
- [] No business decision logic is placed in repositories.
- [] New domains map to dedicated tables (not mixed into unrelated tables).
- [] Database-first workflow is followed for new models/tables.
- [] Constants/enums are used for reusable values; no avoidable magic strings.
- [] Service layer contains application logic.
- [] `PasswordManager.cs` encryption behavior is preserved unless a justified change is explicitly reviewed.
- [] Controllers are thin and delegate complex work.
- [] View models are frontend-focused and practical.
- [] Views are organized one folder per screen.
- [] No additional framework is introduced.